AIRLOCK
GOVERNED ADMISSION

USER MANUAL

Version 0.4.0

Local-First LLM-Assisted Software Engineering

Leon Priest · github.com/7h3v01d · Apache-2.0

1 Introduction

1.1 What Is Airlock?

Airlock is a local-first pipeline platform that governs the use of large language models (LLMs) in software development. It does not replace your LLM — it sits between your LLM and your codebase, ensuring that nothing an AI generates is ever trusted until it has been independently verified, tested, and formally admitted through a governance gate.

The central principle of the platform is:

> **Core Principle** The LLM may propose. The spec constrains. The verifier challenges. The certifier measures. The governance gate decides. The runtime only executes what has been admitted.

1.2 Who Is This For?

- Solo developers and small teams who want to use local LLMs for real development without surrendering codebase control.
- Teams working in regulated or high-stakes environments where every change needs a traceable justification and a human decision point.
- Platform builders extending trusted systems with AI-generated plugins, adapters, or modules that must be certified before loading.
- The author's own project integrations — Niles, Librarian Pro, Voice Appliance, FileTracker, and others.

1.3 What Airlock Is Not

- It is not an AI coding assistant or agentic developer.
- It is not a test generator or static analysis wrapper.
- It is not a CI/CD plugin.

It is all of those things inside a trust pipeline that none of them individually provides.

1.4 The Evidence Chain

For every admitted artefact, Airlock provides a complete, tamper-evident record answering:

Question	Evidence Document
Why does this file exist?	Linked to spec_id
What model generated it?	generation_manifest.json

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

Question	Evidence Document
What did the verifier find?	static_verification.json
What tests certify it?	test_report.json
What policies did it pass?	policy_report.json
Who admitted it?	gate_decision.json
What is its certification level?	admission_certificate.json

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

2 Installation & Requirements

2.1 System Requirements

Component	Requirement
Python	3.11 or later
Operating System	Windows, macOS, or Linux
Ollama (LLM commands only)	Local installation with deepseek-coder-v2:16b pulled
pytest + pytest-cov	Required for the test certifier
FastAPI + uvicorn	Required for the dashboard (optional)

2.2 Installing the Core

From the project root, install the package in editable mode and the test certifier dependencies:

```
pip install -e .
pip install pytest-cov pytest-json-report
```

2.3 Installing the Dashboard (Optional)

The read-only web dashboard is an optional extra. Install it with:

```
pip install -e ".[dashboard]"
```

2.4 Configuring Ollama (Optional)

LLM-assisted commands require Ollama running locally with the recommended model pulled. To set this up:

```
# Install Ollama from https://ollama.ai
ollama pull deepseek-coder-v2:16b
```

> **NOTE** All LLM features are optional. The core pipeline — verify, certify, gate, admit — works completely offline without Ollama.

2.5 Windows Convenience Scripts

The repository includes a set of .bat scripts for Windows users:

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

Script	Purpose
setup.bat	First-time environment setup
run.bat	Launch the CLI
test.bat	Run the full test suite
build.bat	Build a distributable package
doctor.bat	Diagnose environment issues
pip.bat	Install packages into the project environment
shell.bat	Open a shell inside the project environment
sync.bat	Sync dependencies

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

3 Core Concepts

3.1 The Pipeline

Everything in Airlock flows through a single directed pipeline. A candidate cannot skip stages or bypass gates. The pipeline has two entry points — LLM-assisted and manual — that converge at the verify stage:

```
airlock spec draft  →  review  →  airlock spec lock  ─┐
airlock spec new    →  edit    →  airlock spec lock  ─┤
                                                      ├→  airlock generate / airlock
candidate register
                                                      └→  airlock candidate hash
                                                           → airlock verify
                                                           → airlock certify
                                                           → airlock gate
                                                           → airlock admit
                                                             ↑ airlock explain (advisory,
any stage)
```

3.2 Specs

A spec is a structured, hash-locked requirement document. It defines what a candidate must do, what it must not do, and what constitutes acceptable behaviour. The spec exists before the candidate and is never revised to match a failing candidate.

Once locked, a spec receives a SHA-256 hash. Any attempt to modify a locked spec is detected. All candidates generated from a spec carry a reference to its hash, creating a permanent link between requirement and implementation.

3.3 Candidates

A candidate is a quarantined software artefact. All generated code begins in quarantine with zero execution authority. It cannot modify the trusted tree, cannot be imported by the trusted runtime, and cannot be executed until it has passed every pipeline stage.

Candidate status transitions through the pipeline:

Status	Meaning
QUARANTINED	Registered but not yet processed
HASHED	Source tree hash recorded — any subsequent modification is detectable

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

Status	Meaning
VERIFIED	Passed static analysis (all blocking rules)
CERTIFIED	Passed test suite with required coverage
ADMITTED	Governance gate issued ALLOW; artefact copied to trusted tree

3.4 Evidence & Sidecars

Every pipeline stage produces a JSON evidence document. Each document is immediately sealed with a detached SHA-256 sidecar file. The hash is stored outside the file it covers, so no modification of the evidence document can go undetected.

Evidence files for a candidate are stored at:

```
evidence/candidate_0001/
  static_verification.json          # verifier report
  static_verification.json.sha256   # detached sidecar
  test_report.json
  test_report.json.sha256
  coverage_report.json
  coverage_report.json.sha256
  gate_decision.json
  gate_decision.json.sha256
  admission_certificate.json
  admission_certificate.json.sha256
```

COMPATIBILITY Sidecar files are sha256sum-compatible. You can verify any evidence file on any system with: sha256sum --check file.json.sha256

3.5 Certification Levels

Every admitted artefact carries a certification level that describes its execution authority:

Level	Name	Execution Authority
L0	Draft	None
L1	Spec-Aligned	Sandbox only
L2	Verified	Controlled test environment
L3	Test-Certified	Limited trusted use
L4	Governance-Admitted	Full trusted execution
L5	Release-Certified	Full + distributable

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

3.6 LLM Trust Boundaries

The LLM is a speed layer, not a trust layer. These boundaries are enforced architecturally in code, not by convention.

The LLM MAY	The LLM MAY NOT
Draft specifications from user intent	Certify, approve, or admit candidates
Generate candidate implementations	Mark tests as passed or skipped
Generate candidate test suites	Alter evidence logs or reports
Explain verifier failures	Modify the trusted source tree
Suggest remediations	Override policy rules or decide certification

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

4 CLI Reference

All commands are invoked via the airlock entry point. Run airlock --help at any time to see available commands.

4.1 Project Management

Command	Description
airlock init <name>	Initialise a new cleanroom project in a subdirectory named <name>
airlock status	Show an overview of the current project: spec count, candidate statuses, certification summary
airlock report <candidate_id>	Display a full evidence summary for a specific candidate
airlock dashboard [--port 7842]	Start the read-only web dashboard server (default: http://127.0.0.1:7842)

4.2 Spec Commands

Command	Description
airlock spec draft <intent>	LLM drafts a spec.md from a one-line intent string. Requires Ollama.
airlock spec new <description>	Create a new DRAFT spec manually. Opens a spec.md for editing.
airlock spec lock <spec_id>	Hash-seal a spec, transitioning it from DRAFT to LOCKED. Required before generating candidates.
airlock spec list	List all specs with their current status and hash.

4.3 Candidate Commands

Command	Description
airlock generate <spec_id> <module>	LLM generates a candidate implementation and test suite from a locked spec. Requires Ollama.
airlock candidate register <spec_id>	Register a manually-created candidate directory as quarantined.
airlock candidate hash <candidate_id>	Record the SHA-256 hash of the candidate's src/ tree. Must be run before verify.

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

Command	Description
airlock candidate list	List all candidates with their current pipeline status.

4.4 Pipeline Commands

Command	Description
airlock verify <candidate_id>	Run the static verifier. Checks 22 policy rules via AST scanning. Produces static_verification.json.
airlock certify <candidate_id>	Run the test certifier. Executes pytest with coverage in an isolated environment. Produces test_report.json and coverage_report.json.
airlock gate <candidate_id>	Run the governance gate. Consumes all evidence, evaluates policy, produces a binding ALLOW / DENY / QUARANTINE / ESCALATE_TO_OPERATOR decision.
airlock admit <candidate_id>	Admit the candidate to the trusted tree. Copies src/ to trusted/src/ and issues an admission certificate. Only succeeds if gate decision is ALLOW.
airlock explain <candidate_id>	Ask the LLM to explain pipeline failures. Advisory only — does not alter evidence or status.

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

5 Workflows

5.1 First-Time Setup

Run the following steps once to set up a new cleanroom project:

```
# 1. Install Airlock
pip install -e .
pip install pytest-cov pytest-json-report

# 2. Create a new project
airlock init myproject
cd myproject

# 3. Verify the project was created
airlock status
```

5.2 LLM-Assisted Workflow (Recommended)

This workflow uses the local LLM to draft the spec and generate the initial candidate. An Ollama instance with deepseek-coder-v2:16b is required.

```
# Step 1: LLM drafts a spec from your intent
airlock spec draft "Build a local file backup CLI"

# Step 2: Review and edit the draft
# Edit specs/spec_0001/spec.md — verify it matches your requirements

# Step 3: Lock the spec (hash-seal it)
airlock spec lock spec_0001

# Step 4: LLM generates a candidate
airlock generate spec_0001 backup

# Step 5: Record the source hash
airlock candidate hash candidate_0001

# Step 6: Run the pipeline
airlock verify candidate_0001
airlock certify candidate_0001
airlock gate candidate_0001
airlock admit candidate_0001

# If any stage fails, ask the LLM to explain
airlock explain candidate_0001
```

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

5.3 Manual Workflow

Use this workflow when you want to write the candidate code yourself, or when Ollama is not available.

```
# Step 1: Create a spec manually
airlock spec new "Build a local file backup CLI"
# Edit specs/spec_0001/spec.md

# Step 2: Lock the spec
airlock spec lock spec_0001

# Step 3: Register a candidate
airlock candidate register spec_0001
# Place your source code in candidates/candidate_0001/src/
# Place your tests in candidates/candidate_0001/tests/

# Step 4: Hash and run the pipeline
airlock candidate hash candidate_0001
airlock verify candidate_0001
airlock certify candidate_0001
airlock gate candidate_0001
airlock admit candidate_0001
```

5.4 Handling Pipeline Failures

When a candidate fails at any stage, the correct remediation path is:

- Identify the failing rule or test from the evidence document.
- Fix the source code in candidates/<candidate_id>/src/.
- Re-run airlock candidate hash to record the updated hash.
- Re-run the failing stage and all subsequent stages.
- Do not attempt to edit evidence files or sidecars — the gate will detect tampering and deny admission.

IMPORTANT Failed checks are pipeline decisions, not advisory warnings. A blocking verifier rule or failed test must be remediated — it cannot be overridden by editing evidence files.

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

6 Static Verifier

The verifier performs AST-based static analysis of every Python source file in the candidate's src/ directory. It enforces 22 policy rules across three severity levels.

6.1 Rule Severity

Severity	Blocking?	Effect on Pipeline
Critical	Yes	Candidate status set to REJECTED. Pipeline halts. Must be remediated before proceeding.
High	Yes	Same as Critical — pipeline halts.
Medium	No	Recorded in evidence as a warning. Candidate may advance. Gate considers these in risk scoring.

6.2 Complete Rule Reference

Rule ID	Severity	Blocking	What It Catches
IMPORT_SUBPROCESS	Critical	Yes	import subprocess
IMPORT_SOCKET	Critical	Yes	import socket
IMPORT_REQUESTS	Critical	Yes	import requests
IMPORT_URLLIB	Critical	Yes	import urllib
CALL_EVAL	Critical	Yes	eval(...)
CALL_EXEC	Critical	Yes	exec(...)
CALL_OS_SYSTEM	Critical	Yes	os.system(...)
FS_PATH_TRAVERSAL	Critical	Yes	../ in string literals
MISSING_SPEC_LINK	Critical	Yes	No spec link in generation manifest
DYNAMIC_IMPORT	High	Yes	__import__(), importlib
IMPORT_PICKLE	High	Yes	import pickle
UNDECLARED_DEPENDENCY	High	Yes	Imports not declared in manifest
FS_FILE_DELETION	High	Yes	os.remove, shutil.rmtree, Path.unlink
ENV_ACCESS	Medium	No	os.environ, os.getenv
FS_SHUTIL_MUTATION	Medium	No	shutil.move, shutil.copytree, shutil.chown
MISSING_TYPE_HINTS	Medium	No	Public functions without type annotations
MISSING_DOCSTRINGS	Medium	No	Public functions without docstrings

Rule ID	Severity	Blocking	What It Catches
MISSING_TESTS	Medium	No	No tests/ directory present

7 Governance Gate

The governance gate is the final checkpoint before admission. It consumes all evidence documents, verifies their sidecars, evaluates policy, and issues a binding decision. The gate cannot be bypassed.

7.1 Gate Decisions

Decision	Meaning	Next Action
ALLOW	All evidence satisfies policy	Run airlock admit
DENY	A blocking failure exists in evidence	Fix the candidate and re-run the pipeline from the failing stage
QUARANTINE	Evidence is incomplete	Produce the missing evidence documents
ESCALATE_TO_OPERATOR	High-risk pattern detected	Operator must review and manually approve before admission

7.2 Admission Guards

Before any file is copied to trusted/src/, three independent guards run:

Guard	What It Checks
1. Status check	Candidate must be in CERTIFIED status. A forged ALLOW decision on an uncertified candidate is blocked.
2. Gate sidecar verification	gate_decision.json is verified against its .sha256 sidecar. An edited or forged gate decision is rejected.
3. Source immutability	The src/ tree hash is recomputed and compared against the hash recorded by airlock candidate hash. Any post-gate modification is caught.

7.3 Admission Certificate

On successful admission, an admission_certificate.json is issued and sealed. It records the candidate ID, spec ID, certification level, gate decision reference, timestamp, and the full evidence manifest. This certificate is the primary artefact for audit and compliance purposes.

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

8 Dashboard

The dashboard is a read-only web interface that provides live observability over the pipeline. It requires the optional dashboard dependencies.

8.1 Starting the Dashboard

```
# Default (localhost only, port 7842)
airlock dashboard

# Custom port
airlock dashboard --port 8080

# Expose on LAN (use with caution)
airlock dashboard --host 0.0.0.0 --port 8080
```

Open http://127.0.0.1:7842 in your browser after starting.

8.2 Dashboard Pages

Page	What It Shows
Overview	Project name, risk class, spec count, candidate status breakdown
Candidates	Full table of all candidates with origin, pipeline status, and evidence summary. Click any row to drill down.
Candidate Detail	Complete evidence chain for a candidate, with per-document sidecar verification status. Tampered evidence is shown in red.
Audit Log	Every gate decision and admission event, most recent first
LLM Calls	Every LLM call with role, model, hashed prompt, token count, and latency

8.3 Security Properties

- Binds to 127.0.0.1 (localhost only) by default — not exposed to the network.
- No POST, PUT, DELETE, or PATCH endpoints exist. The dashboard cannot modify any project data.
- Candidate IDs are validated with a strict regex before any path operation.
- Path traversal is blocked — attempts to access ../specs or /etc/passwd as candidate IDs are rejected.
- All user-derived and LLM-derived strings rendered in the UI are HTML-escaped to prevent XSS.
- Evidence sidecar integrity is verified on every page load. Any tampered evidence is displayed in red.

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

9 Configuration

Each cleanroom project is configured via a airlock.yaml file in the project root. This file is created automatically by airlock init with sensible defaults.

```
project:
  name: myproject
  language: python
  risk_class: medium

local_first:
  allow_network: false
  allow_remote_llm: false   # set true + add base_url for LAN Ollama
  allow_telemetry: false

llm:
  provider: ollama
  model: deepseek-coder-v2:16b
  base_url: http://localhost:11434

certification:
  test_timeout_seconds: 120
```

9.1 Configuration Reference

Key	Default	Description
project.name	(set by init)	Human-readable project name
project.language	python	Source language (currently python only)
project.risk_class	medium	Risk classification: low, medium, or high
local_first.allow_network	false	Permits network access in certified candidates
local_first.allow_remote_llm	false	Permits non-loopback Ollama base_url
local_first.allow_telemetry	false	Enables any telemetry (currently always disabled)
llm.provider	ollama	LLM backend provider
llm.model	deepseek-coder-v2:16b	Ollama model name
llm.base_url	http://localhost:11434	Ollama API URL (loopback enforced by default)
certification.test_timeout_seconds	120	Maximum seconds allowed for the test certifier subprocess

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

9.2 Remote LLM Configuration

By default, the Ollama provider validates that base_url resolves to a loopback address (127.0.0.1, localhost, ::1). To use an Ollama instance on your LAN (for example, a dedicated GPU rig):

```
local_first:
  allow_remote_llm: true

llm:
  base_url: http://192.168.0.163:11434
```

SECURITY NOTE Setting allow_remote_llm: true changes the LLM capability profile — the provider is marked as local: false, network_access: true. This is recorded in all evidence generated during LLM-assisted workflows.

10 Troubleshooting

10.1 Common Issues

airlock command not found

The airlock entry point was not registered. Run pip install -e . from the project root and ensure your Python environment's bin/ or Scripts/ directory is on your PATH.

Ollama is not available

LLM commands require Ollama to be running. Start Ollama, then verify it is accessible:

```
curl http://localhost:11434
```

If you receive a connection error, ensure Ollama is installed and the service is running. LLM commands are optional — the core pipeline does not require Ollama.

OllamaURLError on startup

The configured base_url is not a loopback address and allow_remote_llm is false. Either set allow_remote_llm: true in airlock.yaml, or change base_url to http://localhost:11434.

Admission rejected: hash mismatch

The source tree was modified after airlock gate was run. Re-run airlock candidate hash, then re-run airlock certify, airlock gate, and airlock admit in sequence. Do not modify src/ files between airlock gate and airlock admit.

Gate returns QUARANTINE

One or more evidence documents are missing. Run airlock status and airlock report <candidate_id> to identify which stages have not produced evidence, then re-run those stages.

Test certifier times out

The candidate's test suite exceeded the configured timeout. Increase certification.test_timeout_seconds in airlock.yaml, or investigate why the tests are taking longer than expected.

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0

Evidence integrity verification fails in dashboard

A .sha256 sidecar and its corresponding evidence file do not match. This indicates the evidence file was modified after it was sealed. Re-run the relevant pipeline stage (verify, certify, or gate) to regenerate clean evidence.

10.2 Migrating from Pre-v0.2 Projects

Evidence files created before v0.2 do not have .sha256 sidecars. The gate will return QUARANTINE on these projects. To migrate:

```
airlock verify <candidate_id>
airlock certify <candidate_id>
airlock gate <candidate_id>
airlock admit <candidate_id>
```

Also update airlock.yaml if it still specifies model: qwen2.5-coder — change this to model: deepseek-coder-v2:16b.

10.3 Running Diagnostics

```
# Run the full test suite to verify the installation
pytest tests/

# Run Windows diagnostic script
doctor.bat
```

11 Project Structure

A cleanroom project created by airlock init has the following directory layout:

```
myproject/
  airlock.yaml           # Project configuration
  specs/
    spec_0001/
      spec.md              # Specification document
      spec.md.sha256       # Spec hash sidecar
  candidates/
    candidate_0001/
      manifest.json        # Candidate metadata and source hash
      src/                 # Source code (quarantined)
      tests/               # Test suite (quarantined)
  evidence/
    candidate_0001/
      static_verification.json
      static_verification.json.sha256
      test_report.json
      test_report.json.sha256
      coverage_report.json
      coverage_report.json.sha256
      gate_decision.json
      gate_decision.json.sha256
      admission_certificate.json
      admission_certificate.json.sha256
  trusted/
    src/                   # Admitted artefacts only
  logs/
    audit.jsonl            # Append-only gate and admission log
    llm_calls.jsonl        # Append-only LLM call log
```

12 Platform Doctrine

Airlock is governed by eight laws. These are not guidelines — they are the definition of what the platform is. Decisions that violate these laws are not features, they are defects.

Law	Statement
1 — Generated code is untrusted by default	Any code produced by a language model begins in a quarantined state, regardless of which model or how plausible the output appears. This law has no exceptions.
2 — The specification is the source of truth	The specification exists before the candidate. The candidate is judged against the spec. The spec is never revised to match a failing candidate.
3 — The LLM cannot certify its own output	The model may draft, generate, explain, and suggest. It may not approve, certify, admit, or override policy. These decisions belong to the pipeline.
4 — No candidate enters trusted execution without evidence	Admission requires evidence. There is no fast-path, no override-for-convenience, and no 'just this once.' An artefact with missing evidence is an intrusion.
5 — Every admitted artefact is traceable	For every file in the trusted tree, the platform must be able to answer: which spec, which candidate, which model, which tests, which policies, who admitted it.
6 — A failed check is not a suggestion	Failed checks are pipeline decisions. They may be appealed only through formal remediation: fix the candidate, re-run the pipeline, produce new evidence, re-submit.
7 — Human override is policy-bound and logged	Every override must record who invoked it, which check was overridden, the justification, and the timestamp. This record is immutable and part of the evidence chain.
8 — Local-first means nothing leaves without authorisation	No source code, prompt, candidate, test result, evidence record, or audit log leaves the machine without an explicit, policy-governed, operator-authorised action.

© 2026 Leon Priest · github.com/7h3v01d · Apache-2.0